AWS CLI + GUI DevOps Lab Guide

Overview

- CLI: VPC, Subnets, Route Tables, NACL, Security Group

- GUI: IAM Role, EC2 Instance

- Region: us-east-1

: EC2 Instance (GUI)

1. Go to EC2 Console → Launch Instance

2. Name: DevOps-Server

3. AMI: Ubuntu (e.g., ami-0fc5d935ebf8bc3bc)

4. Instance type: t2.micro

5. Key pair: DevOps-Key

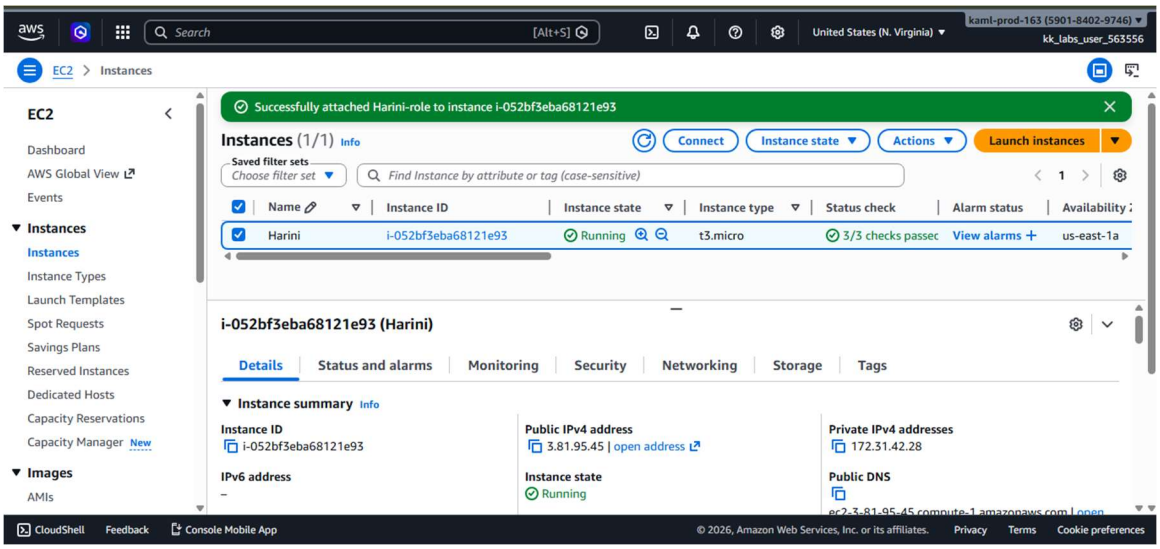
6. Network: Select DevOps-VPC, DevOps-Public-Subnet

7. Security Group: DevOps-SG

8. IAM Role: Attach DevOps-Profile

9. Enable public IP.

10. Launch.



Section 2: IAM Role (GUI)

1. Go to IAM Console → Roles → Create Role

2. Select EC2 as trusted entity.

3. Attach policies:

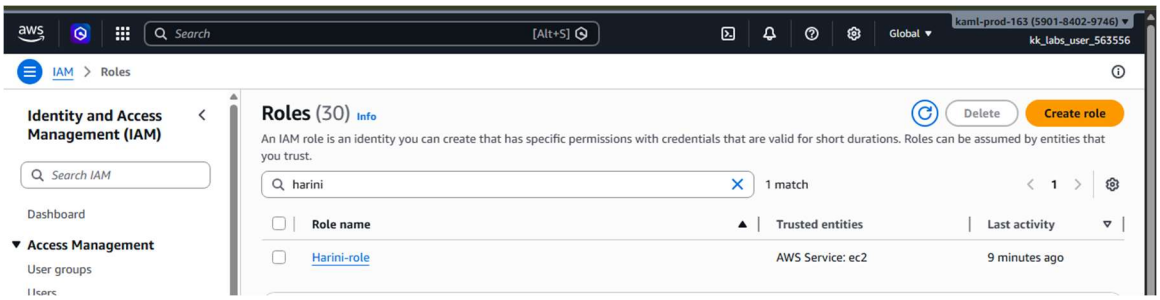
   - AmazonSSMManagedInstanceCore

   - AmazonS3ReadOnlyAccess

4. Name role: DevOps-EC2-Role

5. Create Instance Profile (IAM → Instance Profiles → Create → Add Role).

6. Name profile: DevOps-Profile.



Section 3: Prerequisites

Install AWS CLI

Commands:

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install

```
aws --version
```

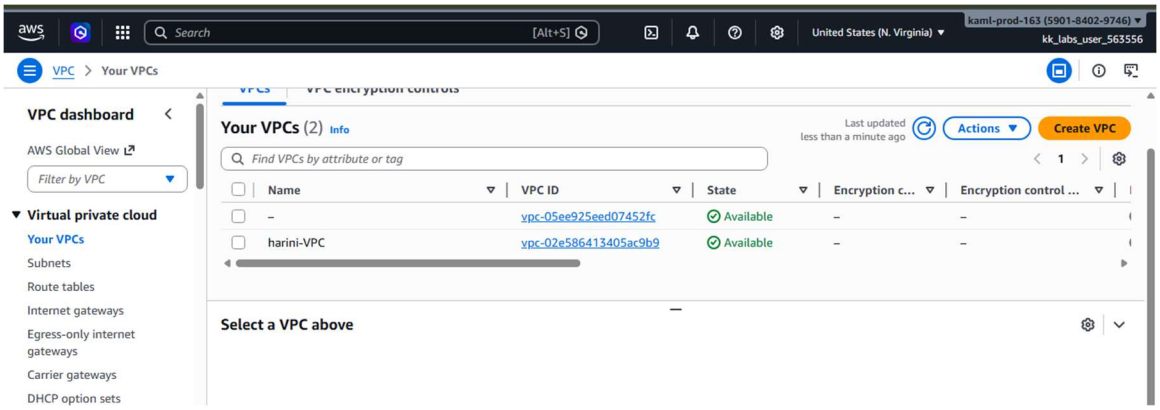
Section 4: Network Setup (CLI)

Variables:

```
REGION=us-east-1
AZ=us-east-1a
VPC_CIDR=10.0.0.0/16
PUBLIC_SUBNET_CIDR=10.0.1.0/24
PRIVATE_SUBNET_CIDR=10.0.2.0/24
```

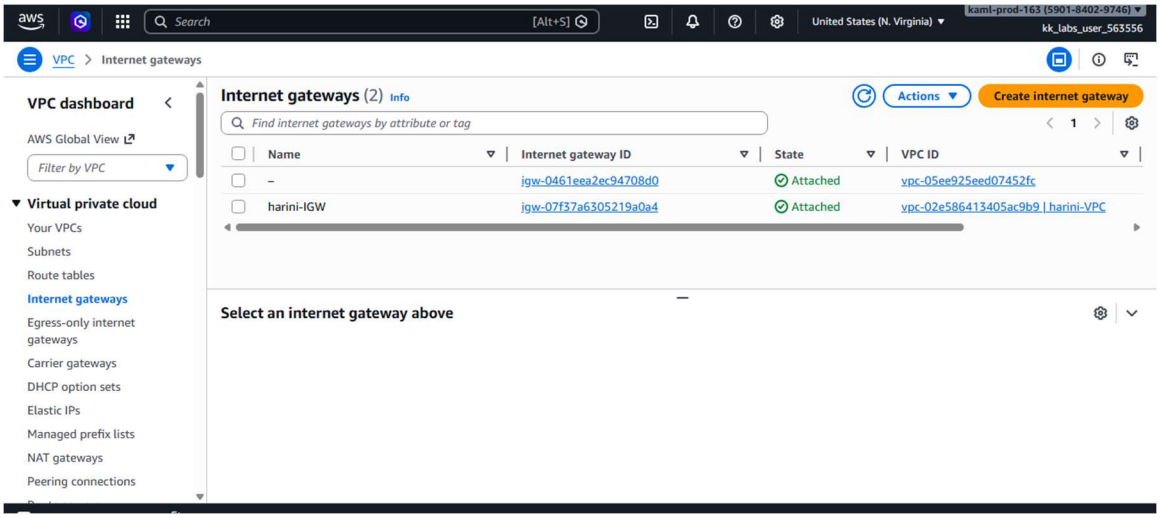
Step 1: Create VPC

```
VPC_ID=$(aws ec2 create-vpc --cidr-block $VPC_CIDR --region $REGION --
query 'Vpc.VpcId' --output text)
aws ec2 create-tags --resources $VPC_ID --tags Key=Name,Value=harini-
VPC --region $REGION
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-support
'{"Value":true}' --region $REGION
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-hostnames
'{"Value":true}' --region $REGION
```



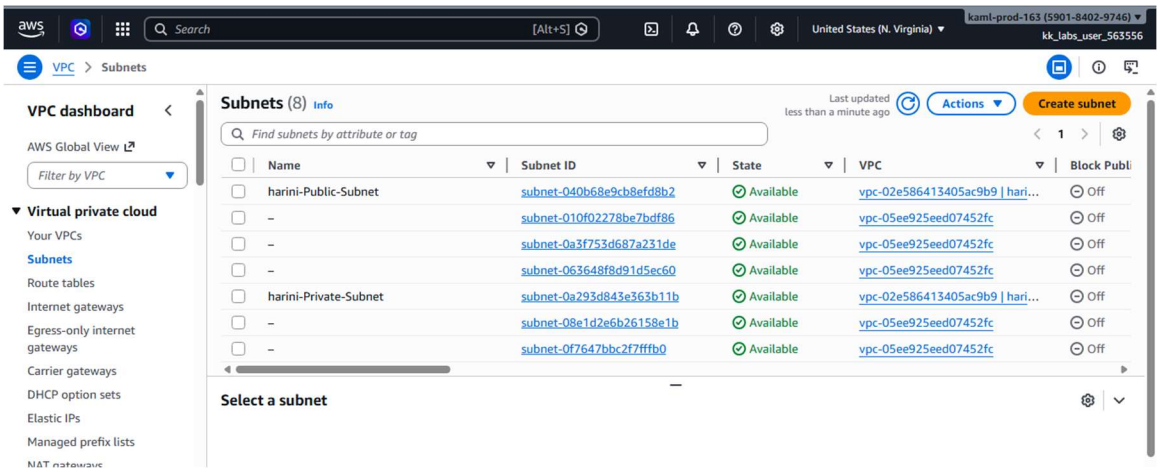
Step 2: Create Internet Gateway

```
IGW_ID=$(aws ec2 create-internet-gateway --region $REGION --query
'InternetGateway.InternetGatewayId' --output text)
aws ec2 create-tags --resources $IGW_ID --tags Key=Name,Value=harini-
IGW --region $REGION
aws ec2 attach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id
$VPC_ID --region $REGION
```



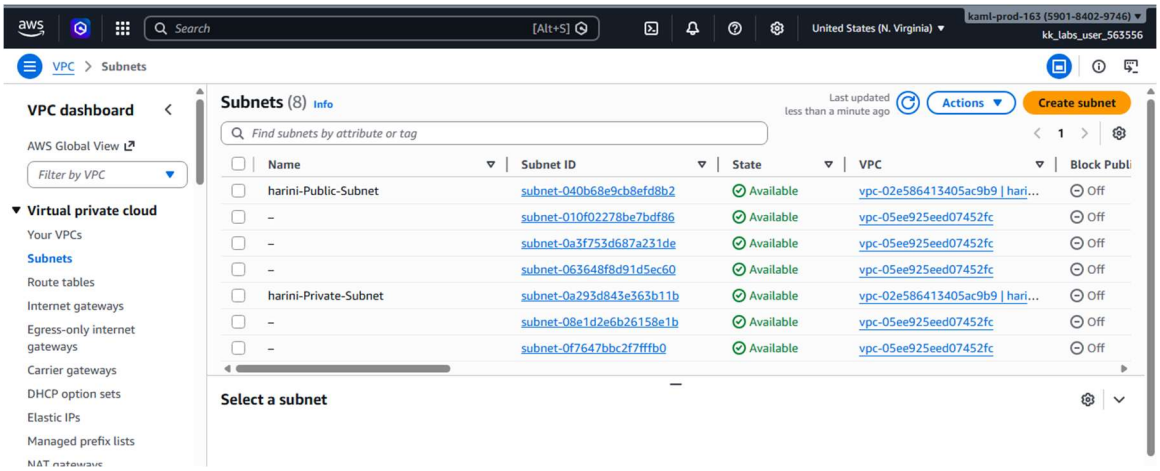
Step 3: Create Public Subnet

```
PUB_SUBNET_ID=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block
$PUBLIC_SUBNET_CIDR --availability-zone $AZ --region $REGION --query
'Subnet.SubnetId' --output text)
aws ec2 create-tags --resources $PUB_SUBNET_ID --tags
Key=Name,Value=harini-Public-Subnet --region $REGION
aws ec2 modify-subnet-attribute --subnet-id $PUB_SUBNET_ID --map-
public-ip-on-launch --region $REGION
```



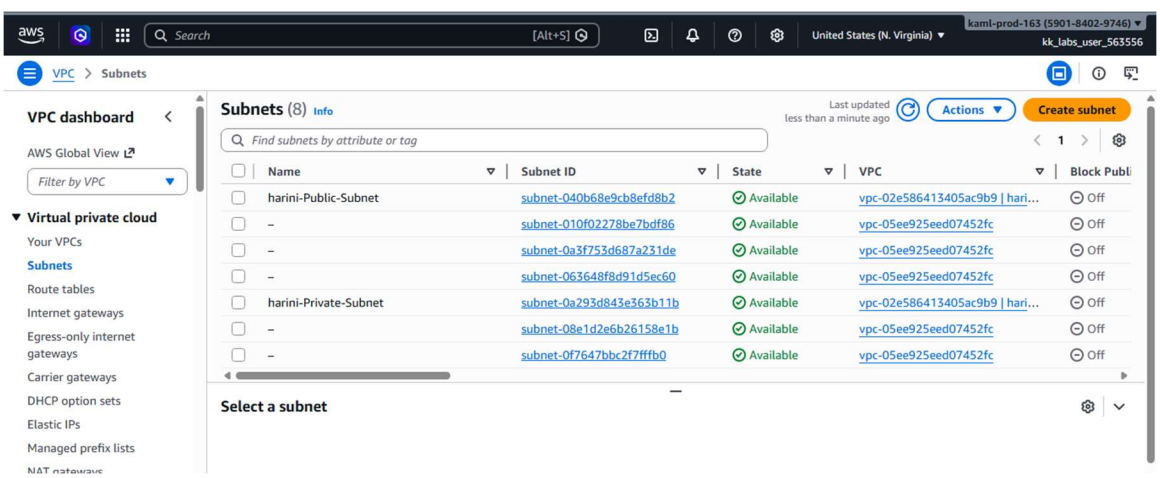
Step 4: Create Private Subnet

```
PRIV_SUBNET_ID=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block
$PRIVATE_SUBNET_CIDR --availability-zone $AZ --region $REGION --query
'Subnet.SubnetId' --output text)
aws ec2 create-tags --resources $PRIV_SUBNET_ID --tags
Key=Name,Value=harini-Private-Subnet --region $REGION
```



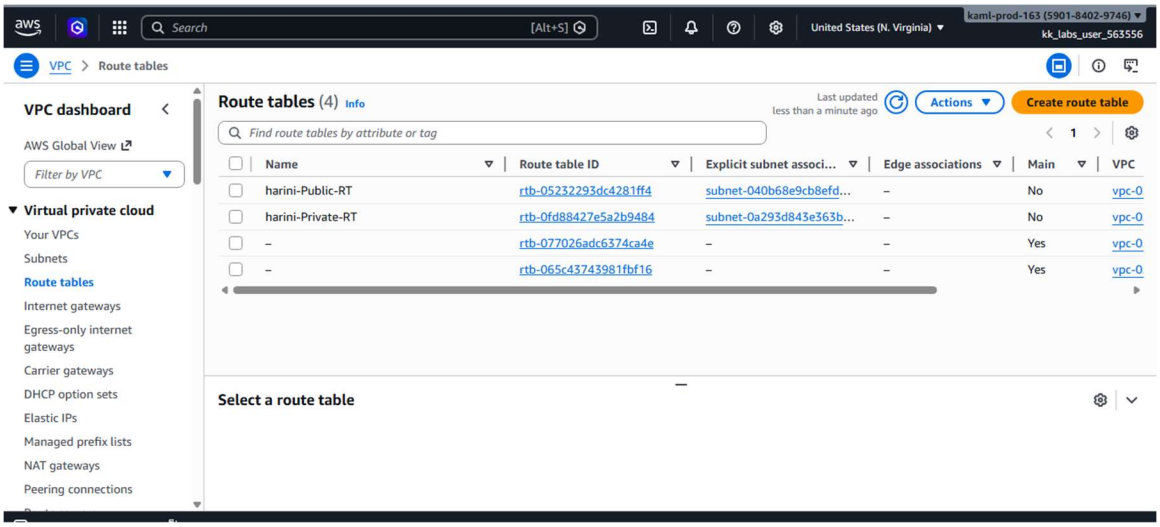
Step 5: Create Public Route Table

```
PUB_RT_ID=$(aws ec2 create-route-table --vpc-id $VPC_ID --region
$REGION --query 'RouteTable.RouteTableId' --output text)
aws ec2 create-tags --resources $PUB_RT_ID --tags Key=Name,Value=harini
-Public-RT --region $REGION
aws ec2 create-route --route-table-id $PUB_RT_ID --destination-cidr-
block 0.0.0.0/0 --gateway-id $IGW_ID --region $REGION
aws ec2 associate-route-table --route-table-id $PUB_RT_ID --subnet-id
$PUB_SUBNET_ID --region $REGION
```



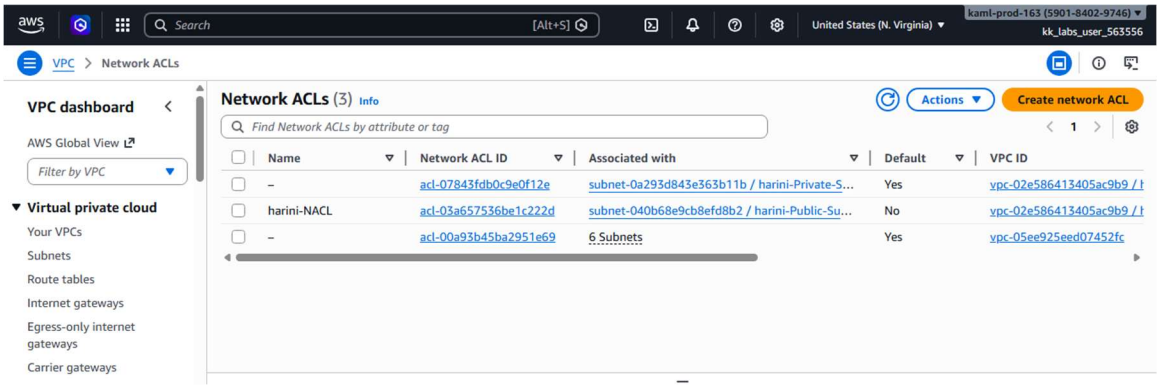
Step 6: Create Private Route Table

```
PRIV_RT_ID=$(aws ec2 create-route-table --vpc-id $VPC_ID --region
$REGION --query 'RouteTable.RouteTableId' --output text)
aws ec2 create-tags --resources $PRIV_RT_ID --tags
Key=Name,Value=harini-Private-RT --region $REGION
aws ec2 associate-route-table --route-table-id $PRIV_RT_ID --subnet-id
$PRIV_SUBNET_ID --region $REGION
```



Step 7: Create NACL

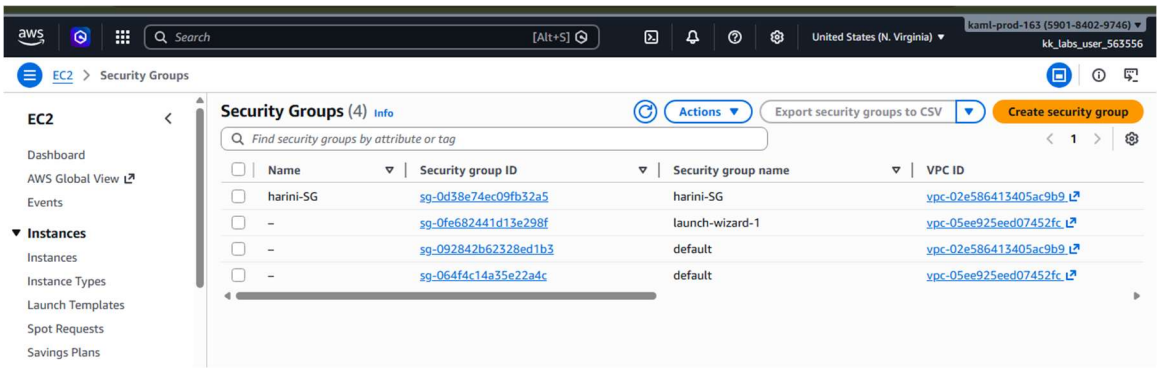
```
NACL_ID=$(aws ec2 create-network-acl --vpc-id $VPC_ID --region $REGION
--query 'NetworkAcl.NetworkAclId' --output text)
aws ec2 create-tags --resources $NACL_ID --tags Key=Name,Value=harini-
NACL --region $REGION
aws ec2 create-network-acl-entry --network-acl-id $NACL_ID --rule-
number 100 --protocol tcp --port-range From=80,To=80 --cidr-block
0.0.0.0/0 --rule-action allow --ingress --region $REGION
aws ec2 create-network-acl-entry --network-acl-id $NACL_ID --rule-
number 110 --protocol tcp --port-range From=22,To=22 --cidr-block
0.0.0.0/0 --rule-action allow --ingress --region $REGION
aws ec2 create-network-acl-entry --network-acl-id $NACL_ID --rule-
number 100 --protocol -1 --cidr-block 0.0.0.0/0 --rule-action allow --
egress --region $REGION
```



Step 8: Create Security Group

```
SG_ID=$(aws ec2 create-security-group --group-name DevOps-SG --
description 'Allow SSH and HTTP' --vpc-id $VPC_ID --region $REGION --
query 'GroupId' --output text)
aws ec2 create-tags --resources $SG_ID --tags Key=Name,Value=harini-SG
```

```
--region $REGION
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol
tcp --port 22 --cidr 0.0.0.0/0 --region $REGION
aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol
tcp --port 80 --cidr 0.0.0.0/0 --region $REGION
```



Section 5: Verification (CLI)

```
aws ec2 describe-vpcs --region $REGION
aws ec2 describe-subnets --region $REGION
aws ec2 describe-instances --region $REGION
aws iam get-role --role-name DevOps-EC2-Role
```

Notes

- Replace AMI if needed.

- Use least privilege policies.

- Restrict SSH CIDR.